

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Algoritmo Pérez

8 de septiembre de 2026

01:57

1. Introducción

La extensión máxima para esta sección es un párrafo breve (máximo 1/4 de página).

Escriba una introducción breve que presente los dos problemas estudiados (ordenamiento de arreglos unidimensionales y multiplicación de matrices cuadradas) y el objetivo del análisis experimental: relacionar el comportamiento práctico de los algoritmos con su complejidad teórica.

2. Entorno experimental

Los experimentos se ejecutaron en un equipo con Windows utilizando WSL2 (Ubuntu), g++ con las opciones `-std=c++17 -O2 -Wall -Wextra`, Python 3, NumPy y Matplotlib. Los programas midieron tiempo en microsegundos y memoria residente en KB mediante `getrusage`. Cada algoritmo recibió una copia del mismo input y se verificó el resultado antes de guardar la medición.

Para sorting se utilizaron los tamaños $n \in \{10, 10^3, 10^5\}$ con los tres tipos de orden y dominios $D1, D7$, tres muestras por combinación, más una entrada representativa $10^7_{\text{aleatorio}, D7, a}$. En total se procesaron 55 entradas. Para matrices se utilizaron $n \in \{2^4, 2^6, 2^8, 2^{10}\}$, los tipos dispersa, diagonal y densa, dominios $D0, D10$ y tres muestras, correspondientes a 72 pares de matrices. Los datos se generan con los scripts de `code/sorting/scripts/` y `code/matrix_multiplication/scripts/`.

3. Resultados y Análisis

Las tablas muestran promedios sobre las muestras disponibles para cada tamaño. Los tiempos están expresados en microsegundos.

Cuadro 1: Tiempo promedio de sorting por tamaño.

n	MergeSort	QuickSort	PatienceSort	<code>std::sort</code>
10	0.78	0.00	1.67	0.44
10^3	46.94	173.50	452.61	11.28
10^5	5315.83	1434692.50	4711136.44	1518.11
10^7	1113552	625490	380219770	680403

La Tabla 1 muestra que `std::sort` es consistentemente competitivo. En 10^5 , QuickSort y PatienceSort empeoran especialmente en entradas con valores repetidos o parcialmente ordenadas. La entrada 10^7 fue el único caso grande medido y PatienceSort tardó aproximadamente 380 segundos. Las Figuras 1 y 2 permiten observar estas tendencias en escala logarítmica.

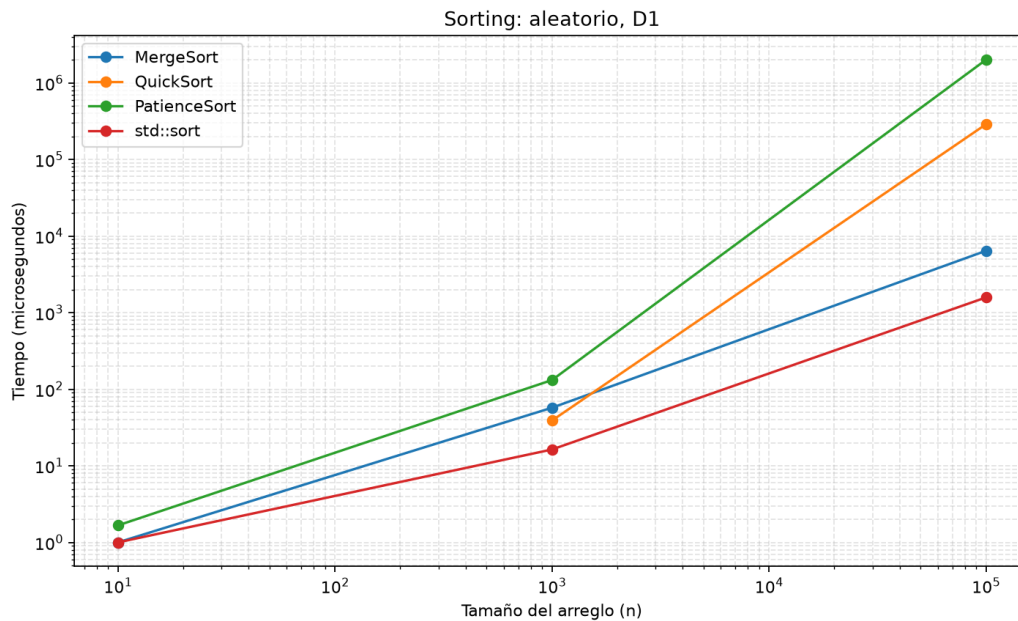


Figura 1: Sorting aleatorio en dominio $D1$.

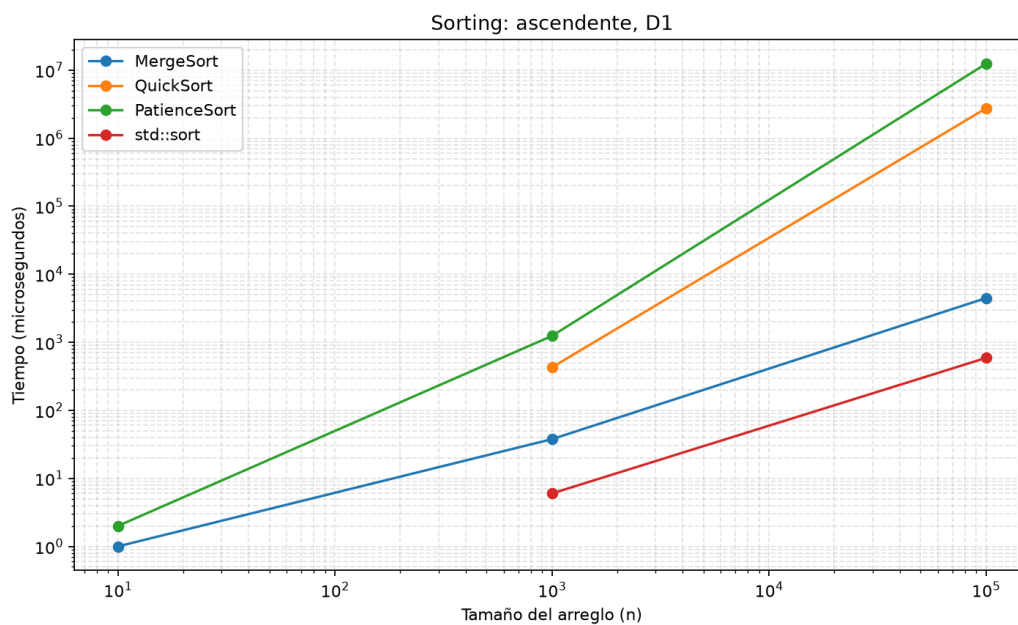


Figura 2: Sorting ascendente en dominio $D1$.

Cuadro 2: Tiempo promedio de multiplicación de matrices.

n	Naive	Strassen
16	1.72	519.11
64	92.50	23323.11
256	7236.06	1150298.44
1024	764579.33	55713463.44

En la Tabla 2, Strassen presenta un costo mayor en este experimento debido a sus constantes, asignaciones y operaciones adicionales. La diferencia es visible en las Figuras 3 y 4; ambas implementaciones produjeron matrices idénticas en los casos procesados.

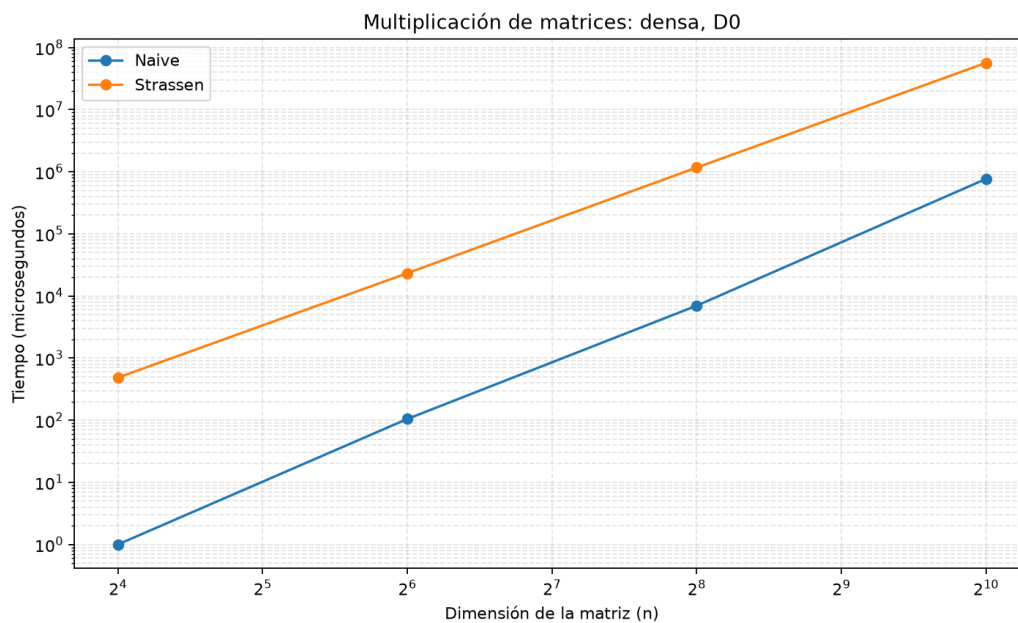


Figura 3: Multiplicación de matrices densas con dominio $D0$.

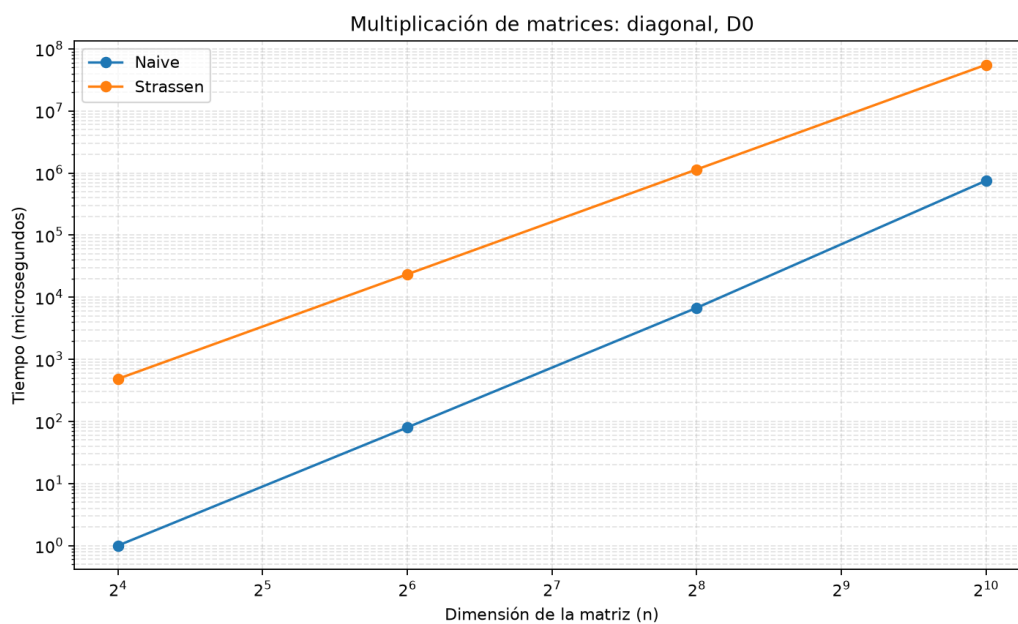


Figura 4: Multiplicación de matrices diagonales con dominio $D0$.

Las visualizaciones se generan automáticamente ejecutando `make plot` en cada directorio. Las figuras utilizadas se encuentran en `code/sorting/data/plots/` y `code/matrix_multiplication/data/plots/`.

4. Conclusiones

Los resultados muestran que el desempeño práctico depende tanto del tamaño como de la distribución de los datos. En sorting, `std::sort` fue el algoritmo más rápido en los tamaños pequeños y medianos, mientras que QuickSort y PatienceSort fueron particularmente sensibles a entradas con muchos valores repetidos o a ciertos órdenes iniciales. En el caso 10^7 medido, PatienceSort alcanzó aproximadamente 380 segundos, frente a 0.68 segundos de `std::sort`. En matrices, Strassen fue más lento que Naive en las dimensiones probadas, aunque ambos entregaron resultados iguales; esto muestra que una mejor complejidad asintótica no garantiza una ventaja para tamaños donde dominan las constantes y la gestión de memoria.